



Git & GitHub for Beginners



Git & GitHub short notes by **Bhanu Prakash Salapu!**



Click the below Triangle Icon (▶) to open the Notes !

▼ 01. Introduction & Setup



What is Git?

👉 Git is a **Version Control System (VCS)** that helps you track changes in your project.



Think of Git like a **time machine for your code**. You can go back to older versions anytime.



Example:

- Without Git → You'll keep 10 folders like `project_v1` , `project_v2` , `final_project_v3` 🤪
- With Git → Just **one project folder** where Git tracks all versions. ✅

What is GitHub?

👉 GitHub is a **cloud platform (Remote)** where you can **store, share, and collaborate** on your Git repositories.

⚡ Git = Local system

⚡ GitHub = Online storage (like Google Drive for code, but smarter)

💡 Example:

- Save your repo on GitHub → Share with friends → They can **contribute**.

Why Learn Git & GitHub?

- ✅ Track changes
- ✅ Work in teams (collaboration)
- ✅ Open-source contributions
- ✅ Industry standard for developers

Installation & Setup

👉 **For Windows:**

1. Go to: <https://git-scm.com/downloads>
2. Download the Windows installer.
3. Open the .exe file and follow the setup wizard.
4. In "PATH environment" option → Select ****Git** from the command line & also from 3rd-party software**.
5. Finish installation.

👉 **For Mac (Homebrew method):**

```
brew install git
```

👉 **For Linux (Ubuntu/Debian):**

```
sudo apt-get update  
sudo apt-get install git
```

✅ **Check installation:**

```
git --version # or  
git -v
```

Example : git version 2.49.0.windows.1

📌 *If you see a version number → Git is installed successfully!*

▼ 02. Git Basics (Local Machine)

📁 **Repository (Repo)** – A folder where Git tracks changes

a) Configure Username & Email (once per machine)

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Check current config:

```
git config --global --list
```



Example:

```
prakash@Prakash MINGW64 ~/Desktop/Git-tuts/practice-git (main)
$ git config --global user.name "Prakash Salapu"

prakash@Prakash MINGW64 ~/Desktop/Git-tuts/practice-git (main)
$ git config --global user.email "prakash.salapu@gmail.com"

prakash@Prakash MINGW64 ~/Desktop/Git-tuts/practice-git (main)
$ git config --global --list
user.email=prakash.salapu@gmail.com
user.name=Prakash Salapu
credential.helper=manager
```

b) Check whether Git is initialized or not

Inside your project folder:

```
ls -a      # shows hidden files like .git (Git Bash/macOS/Linux)
# or (Windows CMD)
# dir /a
```



If you see a `.git/` folder → Git is **initialized**. If not → run `git init`.

```
prakash@Prakash MINGW64 ~/Desktop/Git-tuts/practice-git (main)
$ ls -a
./ ../ .git/ jobs.html main.js style.css

prakash@Prakash MINGW64 ~/Desktop/Git-tuts/practice-git (main)
$
```

c) `git status` (and its file types)

- **Untracked** → New files not tracked yet (add with `git add`).
- **Modified** → Tracked files changed but not staged.
- **Staged** → Changes added to staging area (`git add`), ready to commit.
- **Unmodified** → No changes since last commit.

Check status:

```
git status
```

▼ 03. Clone, Add, Commit & Push - Workflow

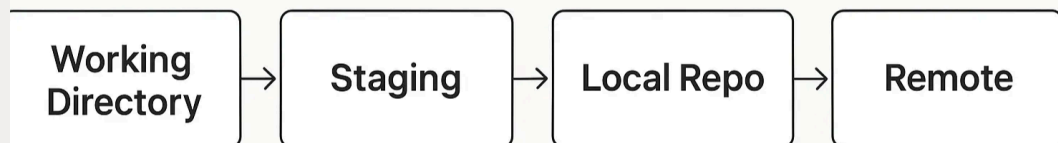
Clone an existing repo

```
git clone <repo-url>  
cd <repo-folder>
```

Daily workflow



Basic Workflow :



```
git status          # see what changed
git add <file>      # stage one file
git add .           # stage everything
git commit -m "meaningful message"
git push origin main # send commits to GitHub
```



Example:

```
prakash@Prakash MINGW64 ~/Desktop/Git-tuts/git-clone-demo
• $ git clone https://github.com/ahmadjaved79/Gallery-App.git
Cloning into 'Gallery-App'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 31 (delta 0), reused 31 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (31/31), 188.16 KiB | 78.00 KiB/s, done.

prakash@Prakash MINGW64 ~/Desktop/Git-tuts/git-clone-demo
• $ cd Gallery-App

prakash@Prakash MINGW64 ~/Desktop/Git-tuts/git-clone-demo/Gallery-App (main)
• $ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```



Tip: Small, focused commits with clear messages make history easy to read.

▼ 04. Git Init - 6 Stages of a Fresh Repo

1. Init repo

```
git init # Starts tracking your project with Git
```

2. Add remote origin (link local → GitHub repo)

```
git remote add origin https://github.com/<user>/<repo>.git
git remote -v
```

#Example:

```
git remote add origin https://github.com/prakashsalapu/Prakash-Portfolio.git
git remote -v
```

#output

```
origin https://github.com/prakashsalapu/Prakash-Portfolio.git (fetch)
origin https://github.com/prakashsalapu/Prakash-Portfolio.git (push)
```

3. Set main as default branch

```
git branch -M main
```

4. Stage files

```
git add .
```

5. Commit snapshot

```
git commit -m "initial commit"
```

6. Push with upstream (so future `git push` works without args)

```
git push -u origin main
```

```
# -u = upstream
```

 Upstream? Links your local main to origin/main. Next time, just git push or git pull.

▼ 05. Branches (Old & New Commands)

Create & switch (old method):

```
git checkout -b new-branch-name
```

#Example:

```
git checkout -b feature1
```

Switch existing branch (old):

```
git checkout feature1
```

Create & switch (new, clearer):

```
git switch -c new-branch-name
```

#Example:

```
git switch -c feature1
```

Switch existing branch (new):

```
git switch feature1
```

List / delete branches

```
git branch          # list local branches
git branch -d name   # safe delete (only if merged)
git branch -D name   # force delete (use with care)
```

|  Best Practice: Create a new branch for every feature/bugfix, then merge.

▼ 06. Merging Code

a) Local Merge (same repository)

```
git switch main  
git merge feature1 # merge feature into main
```

Resolve conflicts if any → commit → push.

b) Pull Request (collaborative / open-source)

1. Push your branch to GitHub.
2. On GitHub → **New Pull Request** (compare your branch → target branch).
3. Add title/description → create PR.
4. Reviewer checks → **Merge**.

| 💡 Use PRs even in solo projects to review your changes before merging.

▼ 07. Git PULL (Bring Remote Changes Locally)

Default (merge)

```
git pull origin main
```

- Fetches & merges remote changes into your current branch (may create a merge commit).

Cleaner history (rebase)

```
git pull --rebase origin main
```

- replays your commits on top of the updated remote branch (no merge commit).

💡 ⚠️ Prefer --rebase for solo work; in teams, follow the repo's policy.

▼ 08. Undoing Git Changes - 3 Common Cases

Case 1: Staged Change - added but not committed

```
git restore --stage <file>    # unstage (keeps changes in working dir)

# older style:
git reset <file>
git reset # for all files
```

Case 2: For last commit (One commit)

```
git reset HEAD-1
```

Case 3: Undo multiple commits

- **Safe (keeps history):**

```
git revert <commitSHA>    # creates a new commit that undoes that c
ommit

#old style :
git reset commit-hash
    #use git.log for commit-hashes

#Example : git reset 82e52219bb7a6642475a132c3ca18e013531563
0 (origin/f1)
```

- **Hard reset (dangerous, rewrites history):**

```
git reset --hard <commitSHA>

#old style:
git reset --hard commit-hash #to delete changes in code also.
```



🔥 Warning: Avoid --hard on shared branches. Use revert instead.

▼ 09. Fork (Copy → Change → PR)

What is Fork?

Your own copy of someone's repo on your GitHub.

Workflow:

1. **Fork** the repo on GitHub (your account gets a copy).
2. **Clone** your fork locally.
3. Create a **branch** → make changes → **commit & push** to your fork.
4. Open a **Pull Request** back to the original repo.

| 💡 Tip: Keep your fork updated by adding the original as upstream:

```
git remote add upstream https://github.com/original/repo.git
git fetch upstream
git rebase upstream/main # or: git merge upstream/main
```

▼ 10. Summary & Recap

Core loop:

status → add → commit → push → pull 

Most used commands:

```
git status
git add .
git commit -m "message"
git push
git pull --rebase
git switch -c <branch> # or: checkout -b
```

Best practices:

- Small commits with clear messages.
- Work on feature branches, not `main`.
- Use PRs to review and discuss changes.
- Avoid force-pushing shared branches.

#Follow for more ..
